LM Voting Simulator

Jiaqiang Zhu

julian25@bu.edu

Repository: https://github.com/SoymilkWinsAgain/LM_Voting_Simulator

Abstract

This project builds and evaluates an LLM-based multi-agent simulator for U.S. presidential election behavior. Each agent is grounded in a real survey respondent profile from the 2024 Cooperative Election Study (CES), optionally enriched with leakage-controlled pre-election survey memory and ANES-derived persona context. Agents are asked to make a hard behavioral choice—vote Democratic, vote Republican, or not vote—and their outputs are evaluated both at the individual respondent level and after state-level aggregation against official election returns from the MIT Election Data and Science Lab. The project began as an attempt to use LLMs as rich-text voter simulators, motivated by the limitations of formal political economy models, conventional polls, and classical machine learning pipelines. The final system is a reproducible, file-based pipeline with CES/ANES/MIT ingest, memory construction, agent prompting, non-LLM baselines, leakage controls, and six evaluation experiments. The main scientific finding is intentionally sobering: the framework works, but the evaluated small local model, qwen3.5:2b, is not yet a reliable voter simulator. It follows the output contract perfectly and uses party/ideology information well, but strict survey memory does not consistently improve performance, candidate context often induces a strong Republican collapse, and aggregate winner accuracy can be misleading. The project therefore contributes less a finished forecasting tool than a rigorous evaluation harness and a set of failure modes for future LLM election-simulation research.

1. Motivation and Problem Statement

Forecasting elections is difficult because voting behavior is simultaneously structured and text-heavy. Formal political economy has long offered elegant models of turnout and vote choice, from rational-choice voting to institutional and spatial theories. Those models are useful for explanation, but their predictive performance is often limited because real voters respond to identities, narratives, issues, candidates, memories, local context, and media environments that are hard to compress into a few equations. Polls are indispensable empirical sources, but they are expensive, not always available at the desired geographic or subgroup resolution, and are themselves observed survey measurements rather than a generative model of voter behavior. Classical machine learning improves predictive flexibility, yet it works best on structured variables and often struggles to use unstructured political text in a natural way.

LLMs suggest a different possibility. They are pretrained on large bodies of political language, can consume natural-language descriptions of a person, and can act as role-playing agents. In principle, this makes them attractive for multi-agent election simulation: instead of only fitting a regression on demographic cells, we can instantiate many voter agents, condition them on survey-derived facts, ask them how they would behave, and aggregate the simulated electorate. This has academic value for studying how much “algorithmic fidelity” LLMs have to real respondents. It also has commercial value: investors want to reason about political risk, campaigns want to evaluate messaging and mobilization strategies, and analysts want fast counterfactual simulations. These use cases create demand for simulators that are cheaper and more flexible than new polls, but also increase the risk of overconfident misuse.

The central question of this project is therefore not simply “Can an LLM predict an election?” It is more precise: can we build an auditable pipeline that turns real survey respondents into LLM voter agents, prevents obvious leakage of post-election labels, compares LLM agents against non-LLM baselines, and identifies where the

simulator fails? The final answer is mixed. The infrastructure succeeds. However, the current small model, i.e. Qwen3.5-2b, does not yet meet the standard for substantive election simulation.

2. Related Work

This project is connected to two research threads. The first is the emerging use of LLMs as synthetic social-science respondents. Argyle et al. (2023) introduced the idea of "silicon samples", showing that GPT-3 conditioned on sociodemographic backstories could reproduce some human survey response distributions. At the same time, Bisbee et al. (2024) warned that LLM-generated survey data can look plausible on averages while failing on variance, subgroup relationships, and prompt sensitivity. Santurkar et al. (2023) similarly showed that LLM opinions can be misaligned with many demographic groups even when prompted to represent them. These papers motivated my insistence on individual-level, subgroup-level, and prompt-robustness evaluations instead of relying on a single aggregate metric.

The second thread is LLM-based political and election simulation. ElectionSim (Zhang et al., 2024) proposes massive LLM-driven voter pools and a poll-based presidential benchmark. Yu et al. (2024) explore large-scale LLM simulation for political decision-making using demographic, temporal, and ideological factors. Parikh, Cen, and Podimata (2026) test whether LLMs track public opinion in the 2024 presidential election and find systematic favorability miscalibration. DEBATE (Chuang et al., 2025/2026) shows that even role-playing agents that seem plausible individually may fail to reproduce group-level opinion dynamics. Together, these works suggest that LLM election simulation is promising but fragile. My project therefore emphasizes controlled evaluation and error analysis over optimistic claims.

This project differs from much of the related work by treating failure analysis as a first-class goal. Rather than constructing only a demonstration, I built a suite of stress tests: shuffled memory, anonymous candidates, candidate-order reversal, state swapping, individual accuracy, aggregate margin error, and subgroup diagnostics.

3. Data Sources

The main respondent source is the 2024 CES Common Content. CES provides a large sample, pre- and post-election waves, rich demographics, party identification, ideology, issue positions, approval questions, turnout and vote self-reports, and vote-validation weights. It is the natural backbone for this version because it contains enough respondents in many states to support both individual and aggregate evaluation.

ANES 2024 was used differently. The initial prototype used ANES respondents as the main personas, but the sample was too small for state-level simulation. In the final design, CES is the primary source, while ANES remains available as a persona donor pool for additional inferred context under strict whitelisting. This preserves ANES's richer political batteries without making it the main evaluation population.

MIT Election Data and Science Lab returns provide the aggregate ground truth. The pipeline uses official presidential returns at the state level, with county-level returns available for future county-aware extensions. The final evaluation thus has two forms of truth: CES post-election respondent labels for individual simulation and MIT official returns for aggregate election outcomes.

All datasets can be downloaded from the official website.

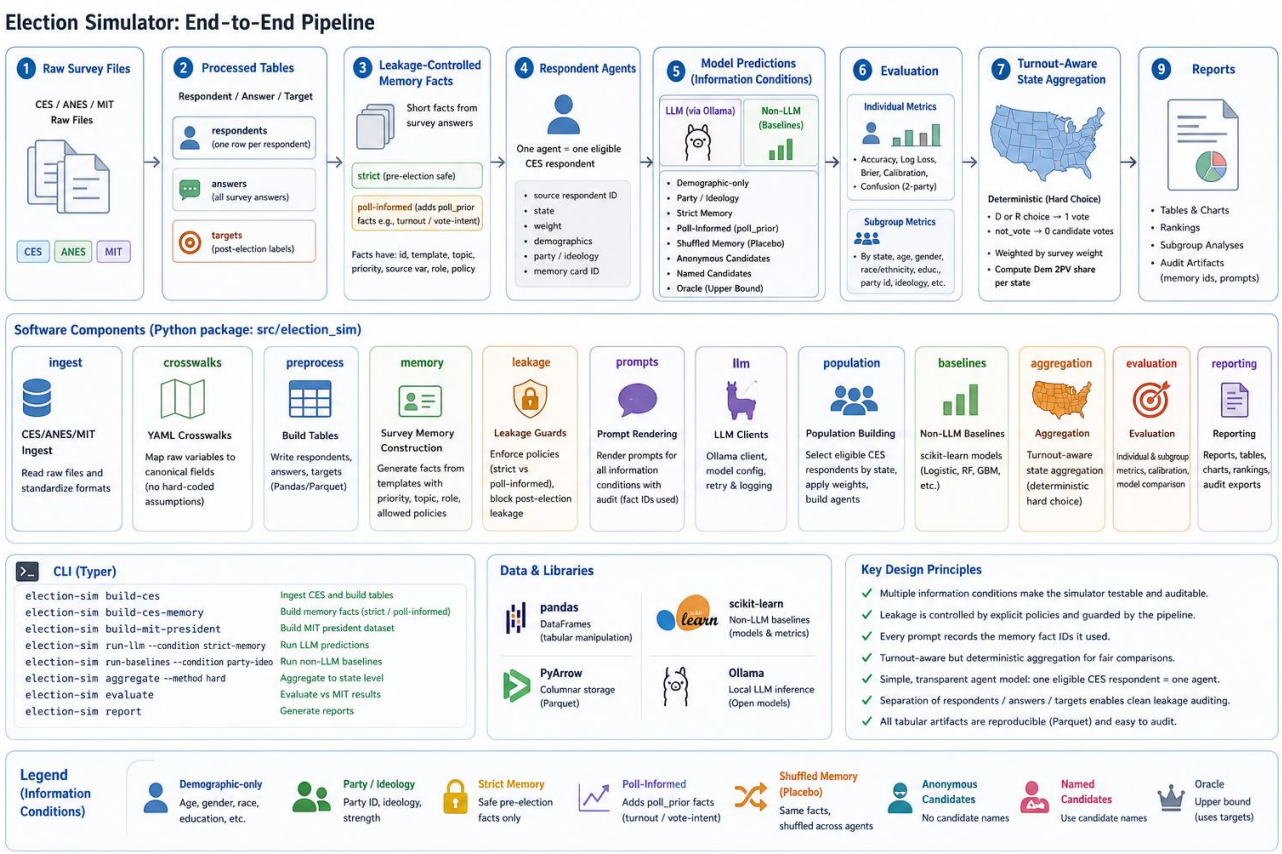
4. System Design and Architecture

This design was intentional: election simulation experiments need reproducibility, auditable artifacts, and the ability to rerun individual stages. Raw data are local-only. Processed artifacts and reports are written as Parquet tables and Markdown reports. The public repository contains source code, configuration files, crosswalks, templates, and test fixtures, but not raw or generated data.

The core pipeline is: raw CES/ANES/MIT files -> processed respondent/answer/target tables -> leakage-controlled memory facts -> respondent agents -> LLM and non-LLM predictions -> individual and subgroup metrics -> turnout-aware state aggregation -> MIT official-result comparison -> reports.

The main software components are Python modules under `src/election_sim`: CES/ANES ingest, survey-memory construction, leakage guards, prompt rendering, LLM clients, population building, baselines, aggregation, evaluation, and reporting. Typer CLI commands expose the stages, such as `build-ces`, `build-ces-memory`, `build-mit-president`, and benchmark runners. The implementation uses pandas/PyArrow for tabular artifacts, scikit-learn for non-LLM baselines, and Ollama for local LLM inference.

Picture 1. Election Simulator End-to-End Pipeline



A key design decision was to use several information conditions rather than a single prompt. The system compares demographic-only prompts, party/ideology prompts, strict memory prompts, poll-informed prompts, shuffled-memory placebos, anonymous-candidate prompts, named-candidate prompts, and oracle conditions. This makes the simulator testable: if adding memory helps, it should beat the simpler party/ideology prompt; if memory is merely generic partisan texture, shuffled memory should perform similarly; if candidate names leak world knowledge, named prompts should behave differently from anonymous ones.

Aggregation is turnout-aware but, in the final hard-choice version, deterministic. A Democratic or Republican choice contributes one vote for that party; a `not_vote` choice contributes zero candidate votes. State-level Democratic two-party share is then computed from the weighted expected Democratic and Republican votes. This made aggregate evaluation comparable across LLM and non-LLM baselines while avoiding the invalid probability-vector problem from the earlier version.

Agent construction is intentionally simple. In the main CES mode, one agent corresponds to one eligible CES respondent selected by state, post-wave completion, citizenship, and sampling configuration. Each agent stores its source respondent ID, state, weight, demographic fields, party/ideology fields, and memory card ID. This makes

individual evaluation direct: the agent's output can be joined back to the respondent's CES post-election target labels.

Survey memory is generated from fact templates. Each template converts a canonical survey answer into a short English statement, such as an economic evaluation or candidate approval fact. Facts carry priorities, source variables, topics, roles, and allowed memory policies. The strict policy keeps only safe pre-election facts; the poll-informed policy explicitly allows direct pre-election turnout or vote-intention facts as `poll_prior` facts. This design makes every prompt auditable: each response stores the memory fact IDs used to create it.

CES preprocessing is driven by YAML crosswalks rather than hard-coded assumptions. The build step reads raw CES variables, maps them into canonical respondent fields such as state, age group, gender, race/ethnicity, education, party identification, ideology, registration status, citizenship, and weights, and separately writes answer rows and target rows. This separation was important because the same raw survey can provide safe pre-election memory facts, forbidden post-election labels, and evaluation-only validated turnout variables. Keeping them in separate tables made leakage auditing much easier.

5. Leakage Control and Response Contract

Election simulation has a serious leakage problem: LLMs may already know historical election results, and survey datasets contain post-election vote labels. I implemented memory policies to separate formal prediction from post-hoc explanation. The strict policy blocks post-election turnout/vote variables, TargetSmart validation fields, and direct pre-election vote-intention variables. Poll-informed memory is allowed as an explicit upper-bound condition but is not treated as leakage-free prediction.

The most important iteration was the response schema. The first design asked the LLM to output a JSON probability vector: turnout probability plus probabilities for Democratic, Republican, other, and undecided. This failed methodologically and empirically. Small LLMs often wrote unnormalized probabilities, and even normalized numbers would not be meaningful calibrated probabilities. The final system instead asks for a hard choice: `{'choice': 'not_vote|democrat|republican'}`. The system then derives one-hot compatibility columns. This change eliminated invalid probability failures and made the task closer to a survey response. It also forced a more honest interpretation: the current simulator is a hard-choice simulator, not a calibrated probabilistic model.

6. Models, Baselines, and Tradeoffs

The main LLM was "Qwen-3.5 2b" served locally through Ollama. This was chosen for practical constraints: the project ran on an RTX 5070 Laptop GPU with 8GB VRAM, so the model needed to support thousands of local calls at low cost. The tradeoff is that the model is small and should be treated as a baseline, not the best possible LLM simulator.

The non-LLM baselines were essential. They included a party-ID rule, demographic-only logistic regression, strict pre-election logistic regression, poll-informed logistic regression, 2020 state prior, uniform national swing, and post-election oracle conditions. These baselines prevent overclaiming. If the LLM cannot beat party/ideology or logistic regression, then richer language prompting has not yet added value.

I also kept oracle and poll-informed conditions even though they are not deployable forecasts. They are diagnostic upper bounds. If an oracle cannot perform well, the response contract or evaluation code is broken. If poll-informed prompts do not improve results, the model is not using even very strong pre-election intention signals.

7. Evaluation Design

I designed six experiments plus a preflight stage. The large suite used ten states: PA, MI, WI, GA, AZ, NV, NC, MN, VA, and CO. This was important because the first swing-state-only evaluation contained only Republican-winning states, making winner accuracy misleading. Adding MN, VA, and CO exposed whether the model could predict

Democratic-winning states. The main metrics were vote accuracy, vote macro-F1, turnout accuracy, state Democratic two-party RMSE, margin mean absolute error, margin bias, winner accuracy, prompt flip rate, and subgroup error. Because outputs are hard choices, probability-style diagnostics were interpreted only as deterministic reliability checks.

Table 1. Evaluation suite overview

Experiment	Purpose	Large-suite scale
E00 Preflight	Validate data contracts, leakage audits, and parse gates	12 smoke LLM calls
E01 Individual	Predict each CES respondent choice	1000 agents, 3 LLM baselines
E02 Aggregate	Aggregate state predictions against MIT truth	10 states, up to 500 LLM agents/state
E03 Robustness	Stress prompt variants	1000 agents x 5 variants
E04 Leakage	Test named, anonymous, masked, and swapped contexts	700 agents x 7 conditions
E05 Ablation	Information ladder and placebo memory	600 agents x 10 baselines
E06 Subgroups	Subgroup and reliability analysis	19,844 reused responses, 0 LLM calls

8. Results

The engineering results were strong. In the large suite, all formal experiments passed quality gates. There were no invalid choices, forbidden choices, legacy probability schemas, or transport errors. E01-E05 made approximately 23,900 LLM calls, while E06 reused prior outputs. The total wall-clock time for E01-E06 large was about 94 minutes, and GPU memory stayed within the 8GB laptop limit.

The scientific results were less favorable to the current LLM. In E01-large, party/ideology was the best LLM condition. It achieved about 0.914 vote accuracy, compared with about 0.871 for strict survey memory and 0.551 for demographic-only prompting. The best logistic baselines performed even better. This means the LLM can use party and ideology, but rich memory did not improve over simpler political identity cues.

In E02-large, aggregate accuracy revealed a major limitation. At sample size 500, `survey_memory_llm_strict` had Democratic two-party RMSE around 0.162, margin MAE around 0.314, and winner accuracy 0.700. It predicted the added Democratic states MN, VA, and CO as Republican. By contrast, logistic baselines had margin MAE around 0.079-0.082. This confirms that winner accuracy alone is dangerous: a model can get many Republican-winning states correct while badly missing margins and Democratic-winning states.

E03-large showed that the hard-choice schema is robust but candidate order still matters. Style and formatting variants had low flip rates, but reversing candidate order caused about a 7.3% choice flip rate and large state margin shifts in some states. Candidate-order balancing should therefore be a default design for future runs.

E04-large gave the most surprising result. Anonymous candidates produced the healthiest distribution: about 53% Democrat and 47% Republican, with much lower margin error. But named candidates, party-only prompts, masked-year prompts, masked-state prompts, and candidate-swap prompts all became strongly Republican-skewed, often above 93% Republican. This is not simple historical-result leakage: if the model were just recalling true 2024 state outcomes, it should not miss MN, VA, and CO. A better interpretation is political-context-induced shortcut behavior.

E05-large clarified the memory question. The full ladder showed that party/ideology (L3) remained the strongest non-oracle LLM condition. Strict memory without candidate context (L5) did contain useful signals, well above shuffled-memory placebos near 0.52. However, adding candidate context hurt badly: party/ideology plus context (L4) fell sharply, and strict memory plus context (L6) also fell. This means memory itself is not useless; rather, the current candidate/context framing overwhelms or distorts the useful memory signal.

E06-large localized the failures. Errors were concentrated among Independent/other respondents, Lean Democrats, Black respondents, and Democratic-winning states. Many weak-information or context-heavy

conditions collapsed toward Republican predictions. Oracle conditions performed near-perfectly, showing that the evaluation labels and schema were not the cause of failure.

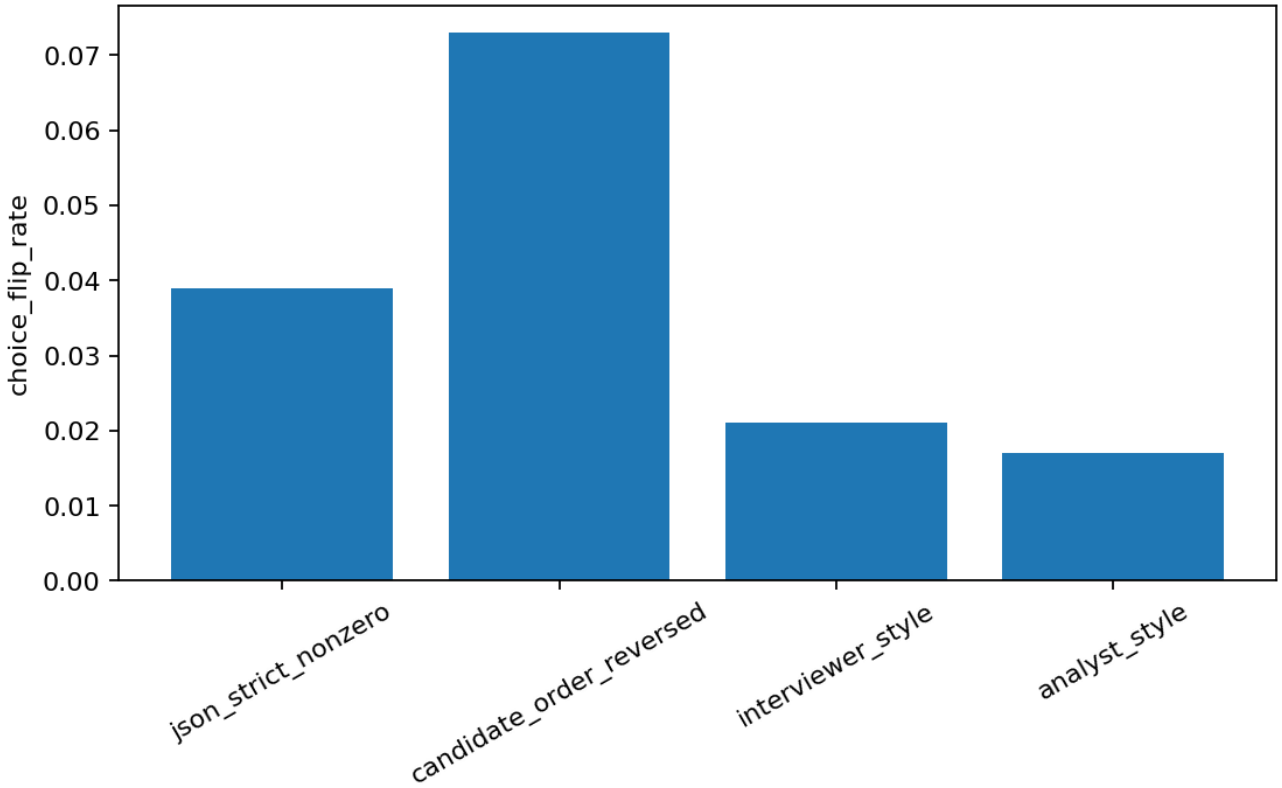
Table 2. E01-large selected individual metrics

Model / baseline	Vote accuracy	Interpretation
Demographic-only LLM	0.551	Weak; demographics alone are insufficient
Party/ideology LLM	0.914	Strongest non-oracle LLM condition
Strict-memory LLM	0.871	Does not improve over party/ideology
Party-ID rule	0.896	Simple non-LLM baseline is competitive
Sklearn strict pre-only	0.974	Structured baseline outperforms LLM
Sklearn poll-informed	0.977	Best individual baseline in headline table

Table 3. E02-large aggregate metrics at sample size 500

Aggregate method	Dem 2P RMSE	Margin MAE	Winner acc.	Interpretation
Survey-memory LLM strict	0.162	0.314	0.700	Strong Republican bias; missed MN/VA/CO
Sklearn pre-only crossfit	0.047	0.082	0.800	Much better margin performance
Sklearn poll-informed	0.048	0.079	0.800	Much better margin performance
Uniform national swing reference	0.015	0.026	1.000	Retrospective reference, not deployable

Picture 2. E03-large choice flip rate



Picture 3. E04-large named vs anonymous candidates delta bars

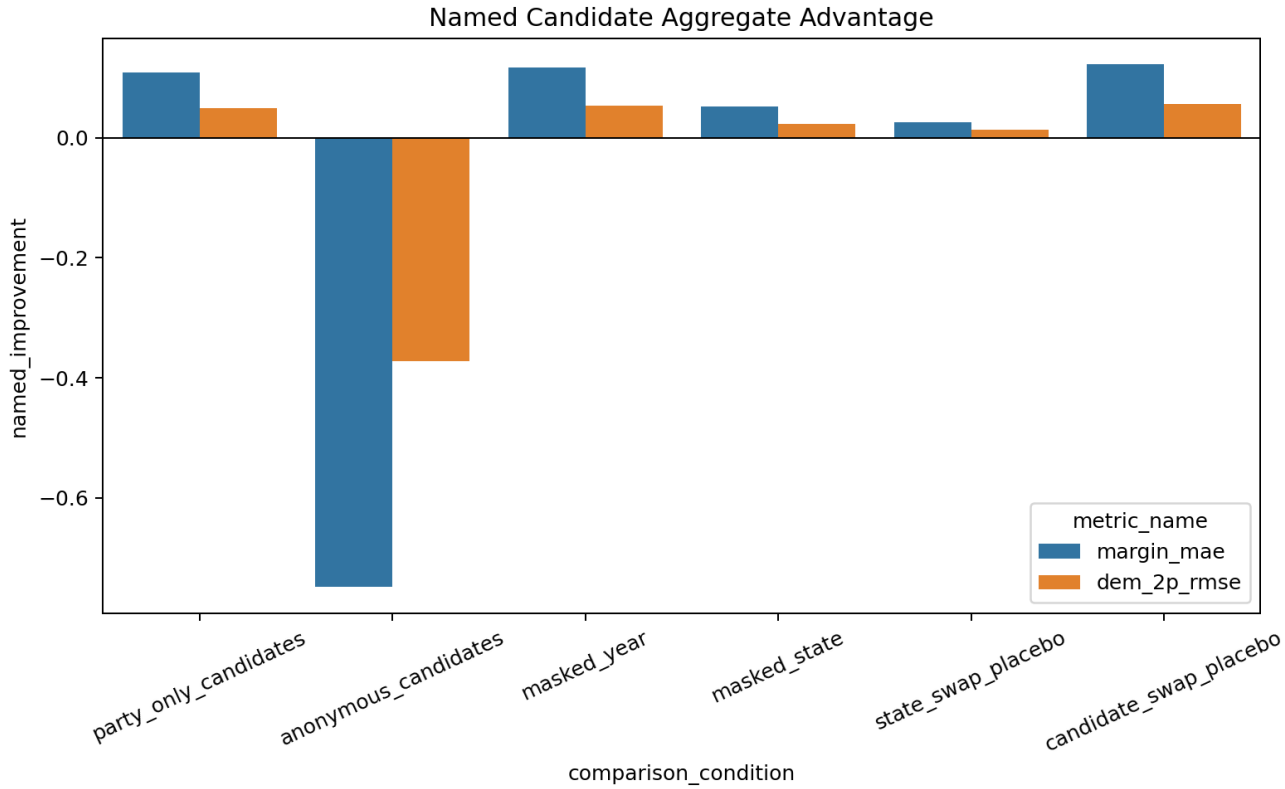
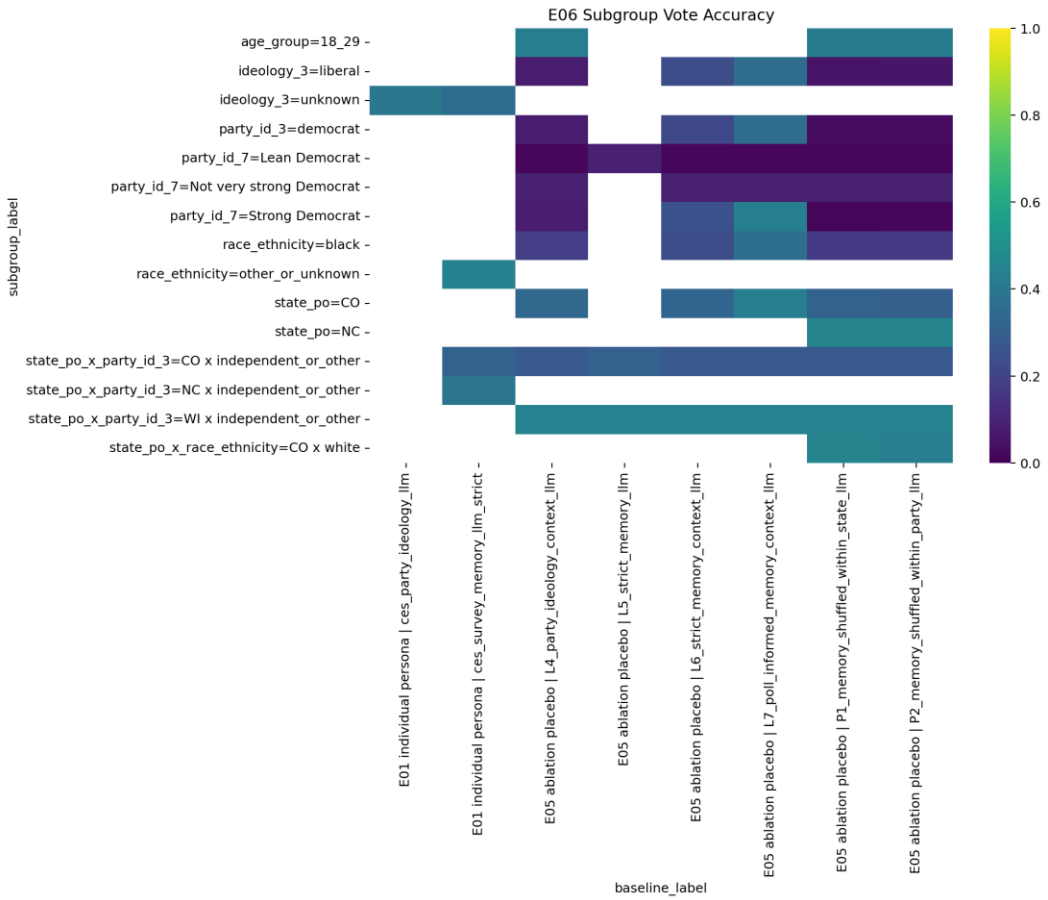


Table 4. E05-large information ladder

Ablation condition	Vote accuracy	Main lesson
L1 demographic-only	~0.512	Little useful signal
L3 party/ideology	~0.893	Best non-oracle LLM condition
L5 strict memory only	~0.806	Memory has real signal
L6 strict memory + context	~0.576	Candidate/context damages performance
P1/P2 shuffled memory	~0.516-0.518	Placebo memory mostly ineffective
L8 post-hoc oracle	~0.987	Schema and labels are not bottlenecks

Picture 4. E06-large subgroup vote accuracy



9. Error Analysis

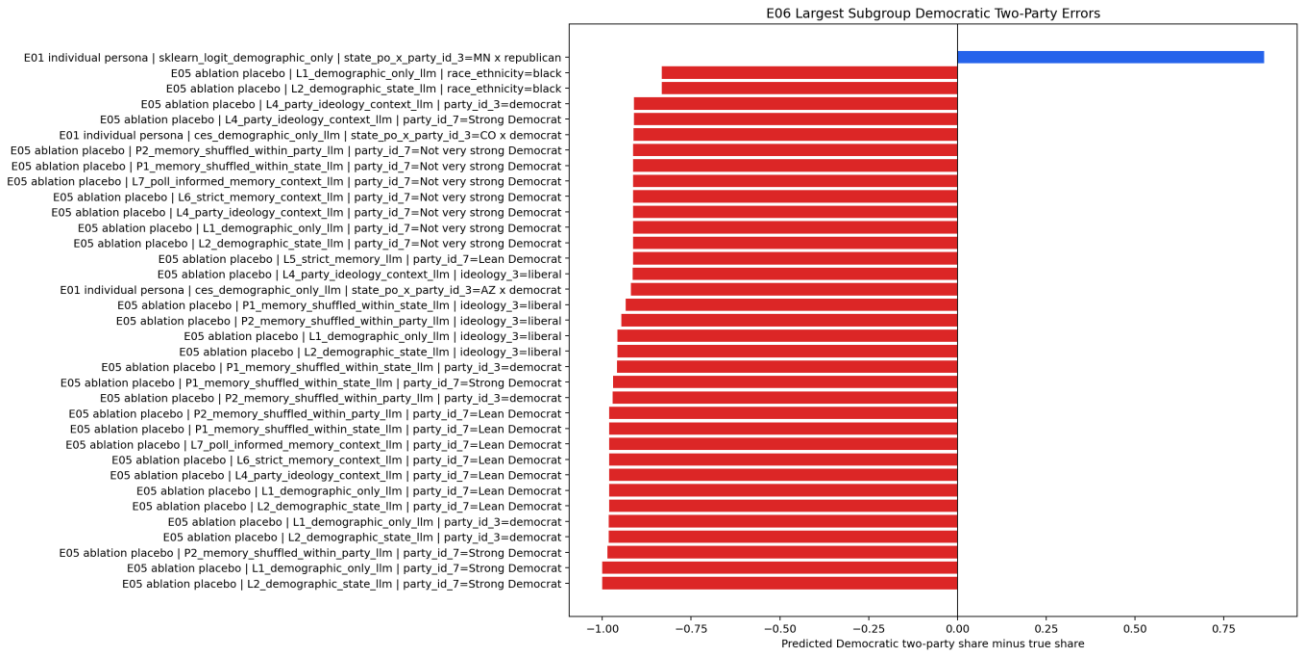
The central failure mode is Republican collapse. When the model lacks clear party/ideology or when election context is added, it often defaults to Republican. This behavior is not a conventional right-wing “truth” model, because it fails in Democratic-winning states. It is better understood as a shortcut triggered by political context, candidate labels, or prompt structure.

A second failure mode is under-modeling nonvoters. The hard-choice schema allows `not_vote`, but the model rarely selects it. Therefore, turnout accuracy is inflated by the voter-heavy sample and should not be read as evidence that the model understands turnout. A better design would separate turnout from vote choice and construct a nonvoter-rich benchmark.

A third failure mode is prompt-order sensitivity. Reversing candidate order changes nontrivial numbers of responses and can shift state margins. This suggests that future aggregate claims should average over balanced candidate orders.

A fourth failure mode is memory misuse. Strict memory contains genuine information, as shown by `L5` beating shuffled placebos, but the model does not reliably combine memory with candidate context. Long natural-language fact lists may be too noisy for a 2B model. Structured memory summaries may work better.

Picture 5. The model collapsed when predicting the Democratic Party.



10. Ethics and Limitations

The project has obvious political misuse risks. A tool that simulates voters could be used for campaign targeting, message testing, manipulation, or investor speculation. Even if the system is inaccurate, users might overtrust confident-looking outputs. For that reason, I would not deploy this version as a forecasting product or campaign tool.

There are fairness concerns. The model underperformed especially for some groups, including Lean Democrats, Independents, Black respondents, and Democratic-winning states. A simulator that systematically misrepresents such groups could reinforce biased political decisions. The reports therefore emphasize subgroup diagnostics, not just average accuracy.

There are privacy concerns even though the data are public survey datasets. The system should never expose individual respondent identities or use restricted raw information. It should present results at aggregate or anonymized levels only.

There are also scientific limitations. The model is small. The current output is deterministic hard choice, not calibrated probability. CES post-election self-reports and weights are themselves survey constructs. The system uses 2024 data after the election, so strict leakage controls are necessary but cannot remove all world knowledge from an LLM. Finally, success on one election would not prove validity for future elections.

11. Iteration and Reflection

The project changed substantially during development. The first prototype used ANES as the main persona source because ANES is rich and well known in political science. That proved too small for the state-level experiments I wanted. I switched to CES as the main respondent population because it has many more cases and better state coverage. Instead of discarding ANES, I kept it as an optional persona donor source with whitelisted variables, which is a better division of labor: CES supplies scale and evaluation labels, while ANES can supply richer context in future versions.

The second major change was the output contract. I initially asked the LLM to output probabilities, but this failed because the model wrote unnormalized and uninterpretable numbers. I replaced that with hard choices. This was

a methodological improvement, not just an engineering patch. It made the system more like survey response simulation and removed a false sense of probabilistic precision.

The final lesson is that negative results are useful. I expected strict survey memory to improve persona fidelity. Instead, the experiments showed that party/ideology dominates, memory alone has limited signal, and candidate context can be harmful. That changed my understanding of the problem. The main contribution became the evaluation framework and the diagnosis of failure modes.

12. Future Work

The next version should test stronger models under the same evaluation suite. It should implement candidate-order balancing by default, redesign memory as structured summaries rather than long fact lists, separate turnout from vote choice, and add repeated sampling or forced-choice token logprobs for calibrated probability estimates. The anonymous-candidate condition should become a formal baseline because it avoided some political-context collapse. Finally, any future report should continue to compare against simple statistical baselines; otherwise, LLM simulation can look more impressive than it is.

The final project version should therefore be evaluated as both a simulator and an evaluation framework. As a simulator, the current local model fails important scientific tests. As an engineering artifact, the project demonstrates reliable data ingestion, leakage-aware prompt construction, controlled baselines, reproducible execution, and honest error analysis. I consider that distinction one of the main lessons of the project.

The reports are not only summaries; they are part of the research artifact. The raw responses, prompts, agent tables, state predictions, subgroup metrics, and Markdown reports remain available under `data/runs/eval_suite_local`. This made it possible to compare the first small suite with the second large suite without rerunning every LLM call. It also makes the project inspectable by another student or an AI coding assistant.

Conclusion

This project tackled a genuinely difficult problem: building and evaluating LLM-driven agents for election behavior. The final system is not a successful forecasting tool, but it is a successful research artifact. It ingests real survey and election data, constructs leakage-controlled agents, runs reproducible experiments, and exposes the limits of a small LLM in a politically sensitive domain. The central finding is that current local LLM agents can imitate simple party/ideology heuristics but do not yet robustly simulate voter personas. That is a valuable result because it tells us where the next scientific and engineering effort should go.

References

American National Election Studies. (2025). ANES 2024 Time Series Study Full Release [Dataset and documentation]. August 8, 2025 version. https://electionstudies.org/data-center/2024-time-series-study/

Ansolabehere, S., Schaffner, B. F., & Pope, J. (2025). Guide to the 2024 Cooperative Election Study. Cooperative Election Study.

Argyle, L. P., Busby, E. C., Fulda, N., Gubler, J. R., Rytting, C., & Wingate, D. (2023). Out of one, many: Using language models to simulate human samples. Political Analysis, 31(3), 337-351. https://doi.org/10.1017/pan.2023.2

Bisbee, J., Clinton, J. D., Dorff, C., Kenkel, B., & Larson, J. M. (2024). Synthetic replacements for human survey data? The perils of large language models. Political Analysis, 32(4), 1-16. https://doi.org/10.1017/pan.2024.5

Chuang, Y.-S., Tu, R., Dai, C., Vasani, S., Li, Y., Yao, B., Tessler, M. H., Yang, S., Shah, D., Hawkins, R., Hu, J., & Rogers, T. T. (2025/2026). DEBATE: A large-scale benchmark for evaluating opinion dynamics in role-playing LLM agents. arXiv:2510.25110. https://doi.org/10.48550/arXiv.2510.25110

Downs, A. (1957). An economic theory of democracy. Harper & Row.

MIT Election Data and Science Lab. (2018/2025). County Presidential Election Returns 2000-2024. Harvard Dataverse. https://doi.org/10.7910/DVN/VOQCHQ

Parikh, R., Cen, S. H., & Podimata, C. (2026). Do LLMs track public opinion? A multi-model study of favorability predictions in the 2024 U.S. presidential election. arXiv:2602.06302. https://doi.org/10.48550/arXiv.2602.06302

Riker, W. H., & Ordeshook, P. C. (1968). A theory of the calculus of voting. American Political Science Review, 62(1), 25-42.

Santurkar, S., Durmus, E., Ladhak, F., Lee, C., Liang, P., & Hashimoto, T. (2023). Whose opinions do language models reflect? Proceedings of ICML. https://proceedings.mlr.press/v202/santurkar23a.html

Yu, C., Ye, J., Li, Y., Li, Z., Ferrara, E., Hu, X., & Zhao, Y. (2024). A large-scale simulation on large language models for decision-making in political science. arXiv:2412.15291. https://doi.org/10.48550/arXiv.2412.15291

Zhang, X., Lin, J., Sun, L., Qi, W., Yang, Y., Chen, Y., Lyu, H., Mou, X., Chen, S., Luo, J., Huang, X., Tang, S., & Wei, Z. (2024). ElectionSim: Massive population election simulation powered by large language model driven agents. arXiv:2410.20746. https://doi.org/10.48550/arXiv.2410.20746

Ansolabehere, Stephen, Brian F. Schaffner, and Jeremy Pope. Cooperative Election Study, 2024: Common Content [Computer File]. Release 2: August 18, 2025. Cambridge, MA: Harvard University [producer]. http://cces.gov.harvard.edu